

DOUBLY WEIGHT-DECOMPOSED LOW-RANK ADAPTATION: DoLoRA

Anonymous authors

Paper under double-blind review

ABSTRACT

Low-Rank Adaptation (LoRA) has emerged as a foundational technique for parameter-efficient fine-tuning (PEFT) of large language models (LLMs). Despite its efficiency, LoRA often exhibits a performance gap compared to full fine-tuning (FFT). Weight-Decomposed Low-Rank Adaptation (DoRA) addresses this limitation by decoupling the merged weight matrix of LoRA into distinct magnitudes and directions. This reparameterization allows the low-rank update to mirror the learning patterns of FFT, aligning gradient trajectories to achieve better optimization. However, DoRA applies this decoupling only after aggregating the matrix with the low-rank update, leaving individual rank-1 components implicitly coupled and poorly conditioned. We propose **DoLoRA**, **D**oubly **L**ow-Rank **A**daptation which decouples magnitude and direction at the component level. By normalizing the low-rank matrices directly, DoLoRA eliminates scale-dependent gradient coupling and reduces computational overhead from $O(dk)$ to $O(r(d+k))$ compared to DoRA, particularly beneficial in the standard $r \ll d, k$ regime. Our results demonstrate that DoLoRA outperforms LoRA and DoRA on downstream supervised fine-tuning (SFT) and mathematical reasoning tasks, notably achieving superior performance in low-rank regimes with greater memory efficiency than DoRA.

1 INTRODUCTION

Large language models have become the foundation of modern AI, but fine-tuning them for specific tasks is expensive in terms of both memory and computation. Low-Rank Adaptation (Hu et al., 2022) (LoRA) offers a solution by enabling parameter-efficient fine-tuning, reducing the number of trainable parameters dramatically. Despite this efficiency, LoRA often struggles to match the performance of full fine-tuning, especially in low-rank settings (Biderman et al., 2024). In practice, this means that for specialized domains, low-rank adapters fail to capture the subtle shifts in data distribution required for optimal performance.

Recent work such as DoRA (Liu et al., 2024) attempts to address this by decoupling the overall magnitude and direction after merging the base weight and the adapter. Because the magnitude is now a standalone parameter, the optimizer can adjust the "strength" of a neuron's output without being constrained by the low-rank direction update. This allows DoRA to capture the complex, non-proportional learning patterns of full fine-tuning that LoRA cannot represent.

Despite these improvements, global decoupling fails to address the fundamental coupling inherent at the level of individual rank-1 components. Each component must internally represent both magnitude and orientation, leading to poor gradient conditioning within the low-rank update. Consequently, the structural expressivity of the adapter remains bottlenecked, particularly in low-rank regimes where precise parameter allocation is critical.

To address these challenges, we propose DoLoRA, a doubly weight-decomposed low-rank adaptation method. DoLoRA decouples magnitude and direction at the level of each rank-1 component, giving each update full flexibility in orientation and scale. It applies weight normalization to both low-rank matrices and learns non-negative scaling coefficients for each component. This removes scale-dependent gradient coupling and leads to better-conditioned optimization, especially in low-rank settings where precise, small updates matter most.

We evaluate DoLoRA on supervised fine-tuning benchmarks and mathematical reasoning tasks, across multiple model sizes, families, and adaptation ranks. DoLoRA outperforms LoRA and DoRA in low-rank regimes, with particularly strong gains on reasoning-heavy tasks such as GSM8K (Cobbe et al., 2021). In higher-rank settings, DoLoRA matches the performance of prior methods while retaining favorable computational and memory efficiency.

In summary, our contributions are:

- *Rank-1 Weight-Normalization*: We introduce DoLoRA, which extends global magnitude–direction decoupling to a component-wise parameterization.
- *Optimization and Conditioning Analysis*: We provide a gradient analysis showing that standard low-rank adaptation suffers from scale-dependent gradient coupling, and that DoLoRA removes this coupling by explicitly separating per-component scale and direction.
- *Performance and Efficiency*: We demonstrate improvements across multiple models, ranks, and reasoning benchmarks, while maintaining favorable computational and memory efficiency.

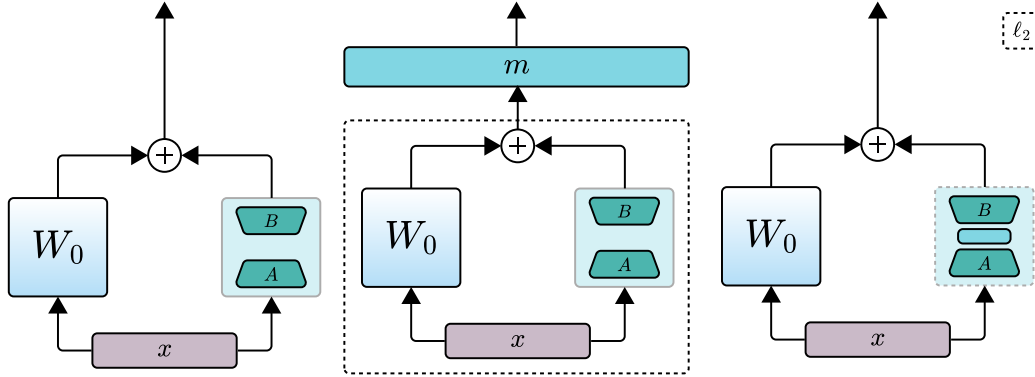


Figure 1: Comparison of magnitude–direction decoupling in low-rank adaptation. LORA (left) represents the update as a low-rank factorization in which the magnitude and direction of each rank-1 component are implicitly coupled. DORA (center) applies weight normalization to the aggregated low-rank update, decoupling scale and direction only at the level of the full adapted weight matrix. DoLoRA (ours, right) instead performs component-wise decoupling by applying weight normalization to the low-rank factors and learning explicit per-component scaling coefficients.

2 RELATED WORKS

Low-Rank Adapters (LoRA) have become a standard approach for fine-tuning large language models (Hu et al., 2022). A growing body of work aims to extend and improve LoRA along several directions, including adaptive rank allocation (Zhang et al., 2023), adapter chaining (Pfeiffer et al., 2020), low-bit training (Dettmers et al., 2023), adaptations for long-context modeling (Chen et al., 2024), and sparsity-inducing methods (Ding et al., 2023).

For this work, LoRA variants that modify the parameterization of the weight update ΔW are particularly relevant. DORA (Liu et al., 2024) reparameterizes the low-rank update using weight normalization (Salimans & Kingma, 2016), decoupling the learning of direction and magnitude. AdaLoRA (Zhang et al., 2023) dynamically allocates adapter rank based on layer importance, effectively learning an SVD-style update with a diagonal matrix and two orthogonal low-rank factors (Zhang et al., 2023). Concurrent work proposes a similar reparameterization of LoRA grounded in techniques from Riemannian optimization (Zhang & Pilanci, 2024). Other approaches, such as VeRA (Kopiczko et al., 2024) and LoRA-XS (Bałazy et al., 2025), introduce alternative parameterizations to further reduce the number of trainable parameters.

A closely related research direction focuses on subspace optimizers, which retain the memory efficiency of low-rank adapters while enabling effective full-rank updates. These methods project

gradients into a low-dimensional subspace where optimizer states are maintained, balancing expressivity and efficiency (Zhao et al., 2024).

3 METHOD

Before introducing DoLoRA, we revisit LoRA (Hu et al., 2022) and DoRA (Liu et al., 2024) to unify notation. This comparison clarifies how existing weight-decomposed methods suffer from gradient coupling issues, which our method is designed to resolve.

3.1 LOW-RANK ADAPTATION (LoRA)

Low-Rank Adaptation (LoRA) is a parameter-efficient fine-tuning (PEFT) method that updates a pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$ by adding a low-rank correction. Rather than directly updating W_0 , LoRA parameterizes the update as

$$W' = W_0 + \Delta W, \quad \Delta W = \frac{\alpha}{r} BA, \quad (1)$$

where $A \in \mathbb{R}^{r \times k}$ and $B \in \mathbb{R}^{d \times r}$ are trainable matrices with rank $r \ll \min(d, k)$, and α is a fixed scaling hyperparameter. During fine-tuning, the original weights W are frozen and only A and B are optimized. In practice, α is typically set proportional to the rank r (e.g., $\alpha = 2r$). In Transformers (Vaswani et al., 2017), applying LoRA to both self-attention and feed-forward blocks seems most beneficial (Schulman & Lab, 2025; Dettmers et al., 2023). The rank r controls the adaptation capacity and induces a trade-off between parameter efficiency and performance.

Initialization of Low-Rank Adaptation (LoRA) To ensure stable optimization and preserve the behavior of the pre-trained model at initialization, LoRA is typically initialized such that the low-rank update is zero in the forward pass. Concretely, one factor (commonly B) is initialized to zero, while the other factor (A) is initialized using a standard random initialization (e.g., Gaussian or Kaiming (He et al., 2015)). This guarantees that $\Delta W = 0$ at initialization, so the adapted model is functionally identical to the pre-trained model while allowing gradients to flow into the low-rank parameters during fine-tuning.

3.2 WEIGHT-DECOMPOSED LOW-RANK ADAPTATION (DoRA)

DoRA (Liu et al., 2024) extends LoRA by introducing a reparameterization that decouples the magnitude and direction of the merged W' matrix. This is motivated by the empirical observation that standard LoRA couples these two components within the same low-rank space, whereas full-parameter fine-tuning (FT) naturally optimizes them with more independence. Specifically, DoRA decomposes the pre-trained weight matrix $W \in \mathbb{R}^{d \times k}$ into a magnitude vector $m \in \mathbb{R}^{1 \times k}$ and a directional component. The resulting weight update is formulated as:

$$W' = m \odot \frac{W_0 + BA}{\|W_0 + BA\|}, \quad (2)$$

where $\|\cdot\|$ denotes the column-wise ℓ_2 -norm and BA is the low-rank directional update. By initializing $m = \|W\|$ and $B = 0$, DoRA ensures the model’s output is identical to the original pre-trained model. This decomposition can be derived by applying weight normalization (Salimans & Kingma, 2016) to the W' matrix in the standard LoRA parametrization. In practice, the gradient calculation of $\|W + BA\|$ is detached from the computational graph to reduce memory and computational overhead (Liu et al., 2024).

3.3 DOUBLY WEIGHT-DECOMPOSED LOW-RANK ADAPTATION (DoLoRA)

We introduce DoLoRA, which extends DoRA by decoupling magnitude and direction at the level of individual rank-1 components within the low-rank update. In contrast, DoRA (Liu et al., 2024) applies weight normalization to the full adapted weight matrix $W + BA$, DoLoRA instead applies normalization directly to the low-rank factors. Specifically, we parameterize the update ΔW as

$$\Delta W = \frac{\alpha}{r} \hat{U} \hat{S} \hat{V}^\top, \quad (3)$$

where $U \in \mathbb{R}^{d \times r}$ and $V \in \mathbb{R}^{k \times r}$ are trainable matrices whose columns are ℓ_2 -normalized:

$$\hat{u}_i = \frac{u_i}{\|u_i\|_2}, \quad \hat{v}_i = \frac{v_i}{\|v_i\|_2}. \quad (4)$$

The diagonal matrix $S = \text{diag}(s_1, \dots, s_r)$ contains learnable non-negative scaling coefficients that control the magnitude of each rank-1 component. Equivalently, the update can be written as a sum of normalized rank-1 factors:

$$\Delta W = \frac{\alpha}{r} \sum_{i=1}^r s_i \hat{u}_i \hat{v}_i^\top. \quad (5)$$

Intuitively, this parameterization restricts U and V to encode directional information, while S independently controls the contribution of each component. Importantly, per-component scaling is necessary to preserve the full expressivity of rank- r updates; normalization without such scaling, or with a single global scale, strictly restricts the class of representable matrices. Specifically, it cannot represent arbitrary rank- r matrices (see Appendix A).

3.3.1 INITIALIZATION OF DoLoRA

Applying LoRA-style zero initialization (e.g., $B = 0$) is not possible in DoLoRA, as the vectors $\hat{u}_i$ and $\hat{v}_i$ become ill-defined when u and v are initialized to zero. Instead, we initialize the vectors u_i and v_i using standard random initializations, e.g., Kaiming initialization (He et al., 2015), ensuring that well-defined unit directions are available from the first optimization step. The overall contribution of the low-rank update is then controlled exclusively through the scaling parameters s_i .

Scaling Parameters To ensure that each rank-1 component has a well-defined non-negative magnitude while preserving meaningful gradients, we reparameterize each scaling coefficient s_i using a shifted softplus function:

$$s_i = \text{softplus}(\lambda_i) - \log(2), \quad \lambda_i \in \mathbb{R}. \quad (6)$$

The softplus parameterization enforces $s_i \geq 0$, consistent with its interpretation as a magnitude, while the constant shift ensures that $s_i = 0$ when $\lambda_i = 0$. By initializing $\lambda_i = 0$, we obtain $\Delta W = 0$ in the forward pass, preserving the identity mapping relative to the pre-trained weights, while maintaining non-zero gradients with respect to both the scaling and directional parameters.

3.4 GRADIENT ANALYSIS

We first analyze the gradients of LoRA and show that its parameterization induces scale-dependent gradient coupling between low-rank factors. This coupling leads to poorly conditioned updates, where the effective step size of one factor depends on the scale of the other. We then show how DoLoRA removes this coupling by explicitly separating scale and direction.

Gradients of Low-Rank Adaptation (LoRA) Let $\mathcal{L}(W)$ be a differentiable loss function under the standard LoRA parameterization $W = W_0 + BA$. By the chain rule, the gradients with respect to the low-rank factors are:

$$\nabla_B \mathcal{L} = (\nabla_W \mathcal{L}) A^\top, \quad \nabla_A \mathcal{L} = B^\top (\nabla_W \mathcal{L})$$

This reveals an inherent *gradient coupling*: the update magnitude for each factor is intrinsically scaled by the current state of the other. Such interdependence often leads to unbalanced optimization dynamics and poor conditioning, particularly when the relative scales of A and B diverge during training.

Gradients of Doubly Weight-Decomposed Low-Rank Adaptation (DoLoRA) By contrast, DoLoRA removes this scale-dependent coupling by explicitly separating per-component scale and direction. Under the DoLoRA parameterization,

$$W = W_0 + \sum_{i=1}^r s_i \hat{u}_i \hat{v}_i^\top, \quad \|\hat{u}_i\|_2 = \|\hat{v}_i\|_2 = 1,$$

the gradients decompose naturally into scale and directional components. Applying the chain rule to the loss $\mathcal{L}(W)$, the gradients with respect to the scaling coefficients satisfy

$$\frac{\partial \mathcal{L}}{\partial s_i} = \left\langle \nabla_W \mathcal{L}, \hat{b}_i \hat{a}_i^\top \right\rangle.$$

Gradients with respect to the raw parameters take the form

$$\begin{aligned} \nabla_{b_i} \mathcal{L} &\propto \frac{s_i}{\|b_i\|_2} \left(I - \hat{b}_i \hat{b}_i^\top \right) (\nabla_W \mathcal{L}) \hat{a}_i, \\ \nabla_{a_i} \mathcal{L} &\propto \frac{s_i}{\|a_i\|_2} \left(I - \hat{a}_i \hat{a}_i^\top \right) (\nabla_W \mathcal{L})^\top \hat{b}_i. \end{aligned}$$

Crucially, the magnitude of the gradient for each factor is controlled by the explicit scale s_i , rather than by the norm of the opposing low-rank factor, eliminating the scale-dependent gradient coupling present in standard LoRA.

This can be interpreted as an orthogonal projection of the gradient onto the tangent space of the unit-norm constraint for each factor. As a result, updates to the directional parameters modify orientation without affecting scale, while magnitude is controlled explicitly through the coefficients s_i . This separation prevents unintended amplification or attenuation of gradients due to factor rescaling.

3.5 COMPUTATIONAL ANALYSIS AND MEMORY

Parameter Count For a weight matrix $W \in \mathbb{R}^{d \times k}$ and rank r , LoRA introduces $r(d + k)$ trainable parameters. DoRA additionally introduces a magnitude vector $m \in \mathbb{R}^d$ under column-wise normalization, incurring an extra $O(d)$ parameter overhead. In contrast, DoLoRA only introduces r learnable scaling coefficients $\{s_i\}_{i=1}^r$. Since $r \ll d$ in typical PEFT settings, the parameter footprint of DoLoRA is effectively identical to LoRA while avoiding the full-dimensional magnitude vector required by DoRA.

Normalization Overhead and Gradient Integrity The primary distinction between DoRA and DoLoRA lies in how normalization is performed and its implications for scaling with model dimension and adapter rank:

- *DoRA Normalization:* DoRA applies column-wise ℓ_2 -normalization to the full adapted weight matrix $W' = W + BA$. This requires computing norms over all k columns of dimension d , introducing an additional $O(dk)$ operation that is independent of the adapter rank. In practice, this full-rank normalization significantly increases memory usage during backpropagation, and existing implementations often detach the normalization term from the gradient graph to reduce VRAM consumption, at the cost of exact gradient flow (Liu et al., 2024).
- *DoLoRA Normalization:* DoLoRA normalizes only the vectors $u_i \in \mathbb{R}^d$ and $v_i \in \mathbb{R}^k$. This results in an additional normalization cost of $O(r(d + k))$, which scales linearly with the adapter rank. As $r \ll \min(d, k)$ in the intended operating regime, this overhead is minimal and enables DoLoRA to retain exact gradients without gradient detachment heuristics.

Table 1 summarizes the additional parameter and computational overhead introduced by each method beyond the shared low-rank update.

Table 1: Comparison of additional parameter and computational overhead beyond the shared low-rank update. DoLoRA eliminates the full-rank $O(dk)$ normalization cost and preserves exact gradients without detachment approximations.

Method	Extra Params	Normalization Overhead	Gradient	Additional Overhead
LoRA	$r(d+k)$	—	Exact	—
DoRA	$r(d+k)+d$	$O(dk)$	Approx. [†]	$O(dk)$
DoLoRA (Ours)	$r(d+k+1)$	$O(r(d+k))$	Exact	$O(r(d+k+1))$

[†]Requires gradient detachment to manage $O(dk)$ VRAM overhead.

4 EXPERIMENTS

We evaluate LoRA (Hu et al., 2022), DoRA (Liu et al., 2024), and DoLoRA across multiple backbone models, sweeping the adapter rank $r \in \{2, 4, 8, 16, 32\}$. Experiments are conducted on LLaMA-3-8B (Grattafiori et al., 2024) and Gemma-3-4B (Gemma Team et al., 2025), each averaged over 3 seeds. By selecting backbones at different scales, we test whether performance trends are robust to model size.

4.1 SUPERVISED FINE-TUNING (SFT)

Following prior work (Hu et al., 2022), we perform supervised fine-tuning (SFT) on a multi-task commonsense reasoning corpus constructed by concatenating the training splits of HellaSwag (HS) (Zellers et al., 2019), PIQA (Bisk et al., 2020), WinoGrande (WG) (Sakaguchi et al., 2021), BoolQ (BQ) (Clark et al., 2019), and CommonsenseQA (CSQA) (Talmor et al., 2019). We do not apply explicit dataset reweighting. All adapters are evaluated independently using the EleutherAI LM Evaluation Harness (Gao et al., 2024), following default evaluation protocols to ensure reproducibility and fair comparison with prior work. We conduct extensive hyperparameter sweeps for each method and report best-performing configurations; full sweep details and experiment-specific settings are provided in Appendix B.2.

Table 2: DoLoRA consistently outperforms LoRA and DoRA across ranks on LLaMA-3-8B, with the largest gains at low ranks, indicating more parameter-efficient adaptation.

Rank	Method	HS	PIQA	WG	BQ	CSQA	Avg.	Δ
$r = 2$	LoRA	83.58	78.83	81.53	57.58	75.60	75.42	—
	DoRA	83.47	79.22	81.14	55.69	75.68	75.04	−0.38
	DoLoRA (ours)	84.05	79.54	85.00	85.67	81.00	83.05	+7.63
$r = 4$	LoRA	83.87	79.65	82.48	63.67	76.74	77.28	—
	DoRA	83.81	79.66	82.72	63.58	76.82	77.32	+0.04
	DoLoRA (ours)	84.21	78.56	85.56	85.48	81.49	83.06	+5.75
$r = 8$	LoRA	83.97	79.38	82.48	77.16	77.48	80.10	—
	DoRA	84.03	79.92	82.24	77.25	77.40	80.17	+0.07
	DoLoRA (ours)	84.64	78.56	85.95	86.36	81.32	83.37	+3.27
$r = 16$	LoRA	83.94	80.31	83.82	84.31	78.13	82.1	—
	DoRA	83.96	80.00	83.82	84.31	79.03	82.2	+0.10
	DoLoRA (ours)	84.61	79.51	86.19	87.86	81.98	83.83	+1.73
$r = 32$	LoRA	84.36	79.76	85.16	84.86	79.77	82.78	—
	DoRA	84.33	79.82	85.32	84.92	79.85	82.72	−0.06
	DoLoRA (ours)	84.61	78.51	86.19	87.80	81.65	83.75	+0.97

Tables 2–3 report supervised fine-tuning results across multiple benchmarks and adapter ranks. We observe that DoLoRA yields improvements over LoRA and DoRA in the low-rank regime across both backbones, while differences narrow as rank increases. These results suggest that the proposed parameterization is particularly effective when adaptation capacity is limited. We note that the differences are most pronounced in BoolQ, CommonsenseQA followed by WinoGrande.

Table 3: DoLoRA yields consistent improvements over LoRA and DoRA on Gemma-3-4B across all ranks, with particularly strong gains in low-rank regimes, highlighting improved parameter efficiency.

Rank	Method	HS	PIQA	WG	BQ	CSQA	Avg.	Δ
$r = 2$	LoRA	78.57	77.97	76.64	63.06	66.18	72.48	–
	DoRA	78.63	78.35	76.56	63.09	65.93	72.51	+0.03
	DoLoRA (ours)	79.70	77.91	81.61	83.67	75.43	79.66	+7.18
$r = 4$	LoRA	78.90	78.51	77.90	75.10	66.75	75.43	–
	DoRA	79.0	78.02	78.45	75.41	67.65	75.71	+0.28
	DoLoRA (ours)	80.0	78.02	82.56	83.73	74.53	79.77	+4.34
$r = 8$	LoRA	79.23	77.80	78.77	76.61	70.03	76.49	–
	DoRA	79.17	78.18	79.40	76.45	70.19	76.68	+0.19
	DoLoRA (ours)	79.86	77.20	81.77	84.10	75.51	79.69	+3.20
$r = 16$	LoRA	79.59	78.46	80.03	80.77	71.50	78.07	–
	DoRA	79.55	78.13	80.0	80.67	71.10	77.89	–0.18
	DoLoRA (ours)	80.0	77.0	82.24	84.13	75.55	79.78	+1.71
$r = 32$	LoRA	79.91	77.64	81.14	82.23	73.22	78.83	–
	DoRA	79.85	78.4	81.14	82.11	73.46	78.99	+0.16
	DoLoRA (ours)	80.28	77.64	83.11	85.0	78.79	80.96	+2.13

Table 4: DoLoRA consistently improves GSM8K accuracy over LoRA and DoRA on both LLaMA-3-8B and Gemma-3-4B, with especially large gains at low ranks, demonstrating superior parameter-efficient reasoning performance.

Rank	Method	LLaMA-3-8B		Gemma-3-4B	
		Accuracy	Δ	Accuracy	Δ
$r = 2$	LoRA	53.5	–	49.5	–
	DoRA	52.0	–1.5	49.2	–0.3
	DoLoRA	61.0	+7.5	51.8	+2.3
$r = 4$	LoRA	61.0	–	53.0	–
	DoRA	54.0	–7.0	53.2	+0.2
	DoLoRA	61.5	+0.5	54.1	+1.1
$r = 8$	LoRA	57.0	–	55.2	–
	DoRA	58.5	+1.5	55.8	+0.6
	DoLoRA	63.5	+6.5	57.4	+2.2
$r = 16$	LoRA	61.2	–	58.4	–
	DoRA	61.5	+0.3	58.9	+0.5
	DoLoRA	65.0	+3.8	61.2	+2.8
$r = 32$	LoRA	62.1	–	59.8	–
	DoRA	62.4	+0.3	60.3	+0.5
	DoLoRA	64.9	+2.8	62.5	+2.7

4.2 MATHEMATICAL REASONING

We evaluate LoRA (Hu et al., 2022), DoRA (Liu et al., 2024), and DoLoRA on GSM8K (Cobbe et al., 2021), a benchmark for multi-step mathematical reasoning. Accuracy is computed by exact match against the gold answer; prompt templates, decoding settings, and additional evaluation details are provided in Appendix B.3. Results are summarized in Table 4. Here, we see that DoLoRA consistently outperforms both DoRA and LoRA across all rank configurations within the GSM8K mathematical reasoning benchmark.

4.3 COMPUTATIONAL EFFICIENCY

We evaluate the computational efficiency of DoLoRA relative to DoRA by measuring the total GPU hours required for a fixed number of training steps, specifically corresponding to the total steps used in our SFT benchmark on LLaMA-3-8B.

As illustrated in Figure 2, while DoRA initially shows a slight performance advantage during the early stages of training, DoLoRA consistently converges to a lower validation loss across all ranks. Ultimately, DoLoRA achieves final convergence approximately 1.33 $\times$ faster than the baseline for a fixed number of training steps.

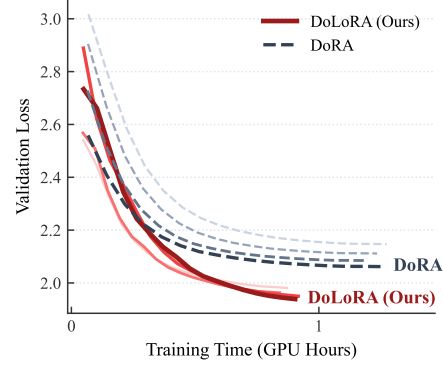


Figure 2: DoLoRA converges faster and reaches lower loss than DoRA, demonstrating improved training efficiency.

5 DISCUSSION AND LIMITATIONS

Scaling at High Ranks. The efficiency of DoLoRA stems from its $O(r(d+k))$ complexity, which is significantly lower than DoRA in the standard $r \ll d, k$ regime. However, this advantage scales linearly with the rank. In extreme cases where r is very large, the computational overhead of component-level decomposition converges toward that of DoRA. While such high ranks are rarely used in practical PEFT applications, DoLoRA is most effective when the rank remains small relative to the hidden dimensions.

Parameter Overhead. Compared to standard LoRA, DoLoRA introduces a marginal increase in trainable parameters due to the inclusion of per-component magnitude scalars. Specifically, for each adapted layer, we introduce r additional parameters to scale the individual rank-1 components. While this overhead is negligible in practice—representing less than 0.001% of the total adapter parameters in a typical $r = 16$ configuration—it represents a slight departure from the minimal parameterization of vanilla LoRA.

Task and Architecture Generalization. Our empirical evaluation focuses on Large Language Models (LLMs) within the domains of supervised fine-tuning and mathematical reasoning. While the theoretical benefits of doubly-decomposed updates should hold for any transformer-based architecture, further empirical study is required to assess DoLoRA’s performance in other modalities, such as Vision Transformers (ViT) (Dosovitskiy et al., 2021) or audio processing. Additionally, investigating the interaction between component-level decomposition and specialized training objectives like RLHF (Ouyang et al., 2022) remains an area for future work.

6 CONCLUSION

In this paper, we introduced DoLoRA, a doubly weight-decomposed low-rank adaptation method that refines the structural parameterization of PEFT. While LoRA provides efficiency and DoRA introduces magnitude-direction decoupling for the aggregated weight, DoLoRA extends this decoupling to the granular level of individual rank-1 components. By explicitly normalizing the low-rank basis vectors and learning per-component scaling coefficients, we ensure that the optimization of directional orientation is mathematically independent of magnitude scaling.

Our theoretical analysis and gradient formulations demonstrate that DoLoRA effectively eliminates the scale-dependent gradient coupling inherent in vanilla LoRA. This leads to a better-conditioned optimization landscape, particularly beneficial in low-rank regimes where precise directional updates are critical for model performance. Furthermore, DoLoRA offers a significant computational advantage over DoRA; by reducing normalization complexity from $O(dk)$ to $O(r(d+k))$, it maintains exact gradient flow without the memory-intensive overhead or detachment heuristics required for full-rank weight normalization. Empirical evaluations across LLaMA-3-8B and Gemma-3-4B backbones confirm that DoLoRA consistently outperforms existing baselines in supervised fine-tuning and mathematical reasoning tasks.

REFERENCES

- Klaudia Balaży, Mohammadreza Banaei, Karl Aberer, and Jacek Tabor. Lora-xs: Low-rank adaptation with extremely small number of parameters, 2025. URL <https://arxiv.org/abs/2405.17604>.
- Dan Biderman, Jacob Portes, Jose Javier Gonzalez Ortiz, Mansheej Paul, Philip Greengard, Connor Jennings, Daniel King, Sam Havens, Vitaliy Chiley, Jonathan Frankle, Cody Blakeney, and John Patrick Cunningham. LoRA learns less and forgets less. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=aloEru2qCG>. Featured Certification.
- Yonatan Bisk, Rowan Zellers, Ronan Le bras, Jianfeng Gao, and Yejin Choi. Piqa: Reasoning about physical commonsense in natural language. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(05):7432–7439, Apr. 2020. doi: 10.1609/aaai.v34i05.6239. URL <https://ojs.aaai.org/index.php/AAAI/article/view/6239>.
- Yukang Chen, Shengju Qian, Haotian Tang, Xin Lai, Zhijian Liu, Song Han, and Jiaya Jia. LongLoRA: Efficient fine-tuning of long-context large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=6PmJoRfdaK>.
- Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In Jill Burstein, Christy Doran, and Thamar Solorio (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 2924–2936, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1300. URL <https://aclanthology.org/N19-1300/>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 10088–10115. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/1feb87871436031bdc0f2beaa62a049b-Paper-Conference.pdf.
- Ning Ding, Xingtai Lv, Qiaosen Wang, Yulin Chen, Bowen Zhou, Zhiyuan Liu, and Maosong Sun. Sparse low-rank adaptation of pre-trained language models. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 4133–4145, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.252. URL <https://aclanthology.org/2023.emnlp-main.252/>.
- Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=YicbFdNTTy>.
- C. Eckart and G. Young. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218, 1936. doi: 10.1007/BF02288367.
- Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. The language model evaluation harness, 07 2024. URL <https://zenodo.org/records/12608602>.

- Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, and Ivo Penchev. Gemma 3 technical report, 2025. URL <https://arxiv.org/abs/2503.19786>.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, and Alex Vaughan. The llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the 2015 IEEE International Conference on Computer Vision (ICCV)*, ICCV ’15, pp. 1026–1034, USA, 2015. IEEE Computer Society. ISBN 9781467383912. doi: 10.1109/ICCV.2015.123. URL <https://doi.org/10.1109/ICCV.2015.123>.
- Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- Dawid Kopiczko, Tijmen Blankevoort, and Yuki Asano. Vera: Vector-based random matrix adaptation. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun (eds.), *International Conference on Learning Representations*, volume 2024, pp. 6815–6835, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/1b53ad08de383a049e9668a9d0b6a053-Paper-Conference.pdf.
- Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. DoRA: Weight-decomposed low-rank adaptation. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (eds.), *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 32100–32121. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/liu24bn.html>.
- Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, Benjamin Bossan, and Marian Tietz. PEFT: State-of-the-art parameter-efficient fine-tuning methods. <https://github.com/huggingface/peft>, 2022.
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022. URL <https://arxiv.org/abs/2203.02155>.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. *PyTorch: an imperative style, high-performance deep learning library*. Curran Associates Inc., Red Hook, NY, USA, 2019.
- Jonas Pfeiffer, Andreas Rücklé, Clifton Poth, Aishwarya Kamath, Ivan Vulić, Sebastian Ruder, Kyunghyun Cho, and Iryna Gurevych. AdapterHub: A framework for adapting transformers. In Qun Liu and David Schlangen (eds.), *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 46–54, Online, October 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-demos.7. URL <https://aclanthology.org/2020.emnlp-demos.7/>.
- Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: an adversarial winograd schema challenge at scale. *Commun. ACM*, 64(9):99–106, August 2021. ISSN 0001-0782. doi: 10.1145/3474381. URL <https://doi.org/10.1145/3474381>.

- Tim Salimans and Durk P Kingma. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. In D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett (eds.), *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016. URL https://proceedings.neurips.cc/paper_files/paper/2016/file/ed265bc903a5a097f61d3ec064d96d2e-Paper.pdf.
- John Schulman and Thinking Machines Lab. Lora without regret. *Thinking Machines Lab: Connectionism*, 2025. doi: 10.64434/tml.20250929. <https://thinkingmachines.ai/blog/lora/>.
- Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. CommonsenseQA: A question answering challenge targeting commonsense knowledge. In Jill Burstein, Christy Doran, and Tamar Solorio (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 4149–4158, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1421. URL <https://aclanthology.org/N19-1421/>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (eds.), *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. Transformers: State-of-the-art natural language processing. In Qun Liu and David Schlangen (eds.), *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45, Online, October 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-demos.6. URL <https://aclanthology.org/2020.emnlp-demos.6/>.
- Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In Anna Korhonen, David Traum, and Lluís Màrquez (eds.), *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 4791–4800, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1472. URL <https://aclanthology.org/P19-1472/>.
- Fangzhao Zhang and Mert Pilanci. Riemannian preconditioned LoRA for fine-tuning foundation models. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (eds.), *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 59641–59669. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/zhang24ax.html>.
- Qingru Zhang, Minshuo Chen, Alexander Bukharin, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. Adaptive budget allocation for parameter-efficient fine-tuning. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=lq62uWRJjiY>.
- Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. GaLore: Memory-efficient LLM training by gradient low-rank projection. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (eds.), *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 61121–61143. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/zhao24s.html>.

A EXPRESSIVE POWER OF DoLORA

We formalize the expressivity of DoLORA in terms of Frobenius norm approximation error. We demonstrate that the per-component scaling mechanism is not merely additional parameters, but a mathematical necessity for maintaining the representational capacity of a standard rank- r update.

A.1 COMPARISON OF SCALING FORMULATIONS

We consider the update matrix $\Delta W \in \mathbb{R}^{d \times k}$ formed by normalized column vectors $\hat{u}_i \in \mathbb{R}^d$ and $\hat{v}_i \in \mathbb{R}^k$. We analyze three potential scaling regimes:

1. **No Scale:** $\Delta W' = \frac{\alpha}{r} \sum_{i=1}^r \hat{u}_i \hat{v}_i^\top$. The magnitude of every component is fixed.
2. **Global Scale:** $\Delta W'' = \gamma \sum_{i=1}^r \hat{u}_i \hat{v}_i^\top$. A single scalar γ controls the total update magnitude.
3. **Per-Component Scale (DoLORA):** $\Delta W''' = \frac{\alpha}{r} \sum_{i=1}^r s_i \hat{u}_i \hat{v}_i^\top$. r independent scalars s_i control the contribution of each rank-1 factor.

A.2 OPTIMAL LOW-RANK APPROXIMATION

Let M be a target matrix (e.g., the ideal gradient update). The Eckart–Young (Eckart & Young, 1936) theorem states that the optimal rank- r approximation in terms of the Frobenius norm ($\min \|\Delta W - M\|_F^2$) is given by the truncated Singular Value Decomposition (SVD):

$$M_r = \sum_{i=1}^r \sigma_i u_i v_i^\top \quad (7)$$

where σ_i are the singular values of M in descending order.

A.3 PROOF OF THE EXPRESSIVITY GAP

We utilize the property that the Frobenius norm of a matrix is equivalent to the ℓ_2 -norm of its singular values. For an optimal reconstruction where the singular vectors of ΔW align with those of M , the error $\mathcal{E}$ is:

$$\mathcal{E} = \sum_{i=1}^r (\sigma_i - \hat{\sigma}_i)^2 + \sum_{j=r+1}^{\min(d,k)} \sigma_j^2 \quad (8)$$

where $\hat{\sigma}_i$ are the singular values provided by the parameterization.

Theorem 1. *The Global Scale variant is a strictly worse approximator than DoLORA for any target update M with non-uniform singular values.*

Proof. In DoLORA, the singular values $\hat{\sigma}_i = \frac{\alpha}{r} s_i$ are independent degrees of freedom. The error is minimized when $s_i = \frac{r}{\alpha} \sigma_i$, yielding the global minimum $\sum_{j=r+1} \sigma_j^2$.

In the Global Scale variant, all singular values are constrained to $\hat{\sigma}_i = \gamma$. The error becomes:

$$\mathcal{E}_{\text{global}} = \sum_{i=1}^r (\sigma_i - \gamma)^2 + \sum_{j=r+1} \sigma_j^2 \quad (9)$$

The term $\sum_{i=1}^r (\sigma_i - \gamma)^2$ is zero **if and only if** $\sigma_1 = \sigma_2 = \dots = \sigma_r$. Because weight updates in deep networks exhibit a non-uniform, often power-law spectrum, σ_i are not identical in practice. Thus, $\mathcal{E}_{\text{global}} > \mathcal{E}_{\text{DoLORA}}$ strictly holds. $\square$

A.4 SUMMARY OF DEGREES OF FREEDOM

As shown in Table 5, DoLORA is the only variant that remains surjective onto the manifold of rank- r matrices. By including r scaling factors, DoLORA restores the "spectral granularity" lost during weight normalization.

Table 5: Representational capacity of scaling variants.

Variant	Learnable Scales	Spectrum Constraint	Expressivity
No Scale	0	$\sigma_i = 1$	Highly Restricted
Global Scale	1	$\sigma_1 = \dots = \sigma_r$	Restricted
DoLoRA	r	None	Full Rank-r

B EXPERIMENTAL SETUP

Experiments are performed on NVIDIA GH200 GPUs. For implementation, we use PyTorch (Paszke et al., 2019), HuggingFace’s Transformers library (Wolf et al., 2020). In particular, we use PEFT: State-of-the-art Parameter-Efficient Fine-Tuning (Mangrulkar et al., 2022) to utilize (Hu et al., 2022) and DoRA (Liu et al., 2024).

B.1 GLOBAL HYPERPARAMETERS

To ensure reproducibility and facilitate a fair comparison across different tasks, we maintained a consistent set of global hyperparameters for all fine-tuning runs. We refer to Table 6.

Table 6: Global Hyperparameters and Training Configuration.

Category	Hyperparameter	Value
Optimization	Algorithm	AdamW
	Weight Decay	0.0
	Alpha (α)	$2r$
Learning Rate	Scheduler Type	Cosine
	Warmup Ratio	0.02
Training	Batch Size	32

To ensure a fair comparison of computational throughput and training latency across all experimental conditions, we avoid the use of gradient accumulation, maintaining a direct 1:1 mapping between physical hardware steps and optimizer updates.

Target Modules To maximize the rank-update coverage across the Transformer (Vaswani et al., 2017) architecture, we targeted all linear projections. Recent work (Schulman & Lab, 2025; Dettmers et al., 2023) indicates that adapting both the self-attention and the multi-layer perceptron (MLP) blocks is critical for capturing the nuances of complex downstream tasks, effectively narrowing the performance gap between LoRA and full-parameter fine-tuning. Hence, across all experiments we apply the adapters on the following layers:

- **Self-Attention Block:** `q_proj`, `k_proj`, `v_proj`, and `o_proj` (Query, Key, Value, Output projections)
- **MLP (Feed-Forward Block):** `up_proj`, `down_proj` and `gate_proj`.

Hyperparameter Tuning To ensure a fair evaluation and prevent overfitting to the test set, we create a validation split for all experiments. We randomly sampled 10% of the training data to serve as a validation set, which was used exclusively for hyperparameter tuning. Using the optimal hyperparameters identified on the validation set, we report the average performance across three independent runs.

For LoRA and DoRA we perform an exhaustive learning-rate sweep for each rank across the grid $\{5 \times 10^{-4}, 1 \times 10^{-4}, 5 \times 10^{-5}, 1 \times 10^{-5}\}$. As mentioned in Section 3.4, DoLoRA decouples the scale-dependent gradient during training leading to much different scale of learning rates. To account for this, we sweep the learning rate across a grid that is much higher, $\{5 \times 10^{-2}, 1 \times 10^{-2}, 5 \times 10^{-3}, 1 \times 10^{-3}\}$. At these learning rates, both LoRA and DoRA suffer from gradient instability leading to divergence.

B.2 SUPERVISED FINE-TUNING (SFT)

We train supervised fine-tuning (SFT) on a multi-task commonsense reasoning corpus constructed by concatenating the training splits of HellaSwag (Zellers et al., 2019), PIQA (Bisk et al., 2020), WinoGrande (Sakaguchi et al., 2021), BoolQ (Clark et al., 2019), and CommonsenseQA (Talmor et al., 2019).

Each adapter is trained on a single epoch of the combined dataset. Empirically, across all adapter types and learning rates, we observe an increase in validation loss leading to overfitting when training on more than one epoch.

During the fine-tuning process, a cut-off length of 512 was applied to all training sequences; any input exceeding this limit was truncated. As mentioned in Appendix B.1, we sweep the learning rates. The final learning rates are reported on Table 7.

Table 7: Learning rate (lr) configurations for backbone models.

(a) LLaMA-3-8B				(b) Gemma-3-4B			
Rank (r)	LoRA	DoRA	DoLoRA	Rank (r)	LoRA	DoRA	DoLoRA
2	1×10^{-4}	1×10^{-4}	1×10^{-2}	2	1×10^{-4}	1×10^{-4}	1×10^{-2}
4	1×10^{-4}	1×10^{-4}	1×10^{-2}	4	1×10^{-4}	1×10^{-4}	1×10^{-2}
8	5×10^{-5}	5×10^{-5}	1×10^{-2}	8	5×10^{-5}	5×10^{-5}	1×10^{-2}
16	5×10^{-5}	5×10^{-5}	1×10^{-2}	16	5×10^{-5}	5×10^{-5}	1×10^{-2}
32	5×10^{-5}	5×10^{-5}	1×10^{-2}	32	5×10^{-5}	5×10^{-5}	1×10^{-2}

B.2.1 EVALUATION

To ensure a rigorous and reproducible comparison, all models were evaluated using the EleutherAI LM Evaluation Harness (Gao et al., 2024).

Following standard practice, we report accuracy based on multiple-choice log-likelihood evaluation. Specifically, we select the candidate with the highest conditional log-likelihood as the predicted answer; where relevant, we apply byte-length normalization to these likelihoods to mitigate potential bias toward shorter answer strings.

B.3 MATHEMATICAL REASONING

To evaluate the model’s capacity for complex logical derivation, we fine-tune on the GSM8K dataset (Cobbe et al., 2021). We ensure consistent performance across the mathematical reasoning tasks, we employ a fixed cut-off length of 2048 tokens. This context window allows the model to process complex, multi-step derivations without truncating essential logical steps.

The training process is conducted over two epochs. This duration was selected to allow for sufficient convergence. Experimental results demonstrate that extending training beyond two epochs consistently yields higher validation loss across all adapter configurations and learning rate schedules, signaling the onset of overfitting.

As mentioned in Appendix B.1, we perform a sweep over the learning rate. We report the final learning rates on Table 8.

Table 8: Learning rate (lr) configurations for backbone models.

(a) LLaMA-3-8B				(b) Gemma-3-4B			
Rank (r)	LoRA	DoRA	DoLoRA	Rank (r)	LoRA	DoRA	DoLoRA
2	1×10^{-4}	1×10^{-4}	1×10^{-2}	2	1×10^{-4}	1×10^{-4}	1×10^{-2}
4	1×10^{-4}	1×10^{-4}	1×10^{-2}	4	1×10^{-4}	1×10^{-4}	1×10^{-2}
8	5×10^{-5}	5×10^{-5}	1×10^{-2}	8	5×10^{-5}	5×10^{-5}	1×10^{-2}
16	5×10^{-5}	5×10^{-5}	1×10^{-2}	16	5×10^{-5}	5×10^{-5}	1×10^{-2}
32	5×10^{-5}	5×10^{-5}	1×10^{-2}	32	5×10^{-5}	5×10^{-5}	1×10^{-2}

B.4 PROMPT TEMPLATE & EVALUATION PROTOCOL

To ensure the model’s conditional generation remains aligned with the fine-tuning distribution, we utilize a consistent prompt template across all phases: `Question:\nq\n\nAnswer:\n`. This setup, along with our answer extraction logic, follows the evaluation protocol established in the original GSM8K benchmark (Cobbe et al., 2021)

For evaluation, the ground truth answer was extracted by isolating the final numerical value immediately following the ‘####’ delimiter present in the standard GSM8K reference strings.